SAL – SIMPLE ACADEMIC LANGUAGE

Manual da linguagem

Leandro Carlos Fernandes

2026 Versão 1.0

Sumário

SAL - *Simple Academic Language* 3
Tipos de dados suportados 3
Declaração de variáveis (simples e vetores) 3
Delimitadores 4
Comentários 5
Palavras reservadas 5
Operadores 5
Ordem de avaliação em expressões 6
Blocos de comandos 7
Comandos de entrada e saída (*scan*, *print*) 7
- Comando PRINT: 7
- Comando SCAN: 8

Comandos de decisão (*if*, *match*) 8
Comandos de repetição (*for*, *loop while* e *loop until*) 10
- Comando FOR 10
- Comando LOOP WHILE 11
- Comando LOOP ... UNTIL 11

Estruturação e seções de um programa em SAL 12
Sub-rotinas (procedimentos e funções) 13
Exemplos de programas em SAL 14
Apêndice A: EBNF da Linguagem SAL 18

SAL – *Simple Academic Language*

Trata-se de uma linguagem de programação didática, simples e minimalista, que dispõe das principais estruturas e elementos comuns as linguagens comerciais.

A linguagem SAL tem por objetivo servir de base no aprendizado da construção de compiladores, projetada para ser fácil de analisar e de traduzir para arquiteturas baseadas em pilha (P-machines).

Podemos descrever a SAL como uma linguagem de característica mista, com estrutura, tipagem, sintaxe e comandos similares as linguagens de programação imperativas como Pascal e C.

Nas subseções que se seguem conheceremos mais sobre a linguagem e trataremos de seus vários elementos constituintes separadamente.

Tipos de dados suportados

Dispomos apenas de três tipos primitivos para o armazenamento de dados, sendo eles: `int`, `bool` e `char`, destinados respectivamente para valores inteiros, lógicos e caracteres simples. Não há uma especificação de tipo para as strings, uma vez que estas são entendidas como vetores de caracteres cujo comprimento varia de 1 e 255 caracteres.

Declaração de variáveis (simples e vetores)

A linguagem SAL especifica apenas variáveis homogêneas, quer sejam variáveis simples ou compostas (vetores), embora não sejam suportados arranjos multidimensionais (matrizes).

Sintaxe:

```
<identificador> : <tipo>;
<identificador>, <identificador>, ..., <identificador> : <tipo>;
```

Dado que a sintaxe determina que a especificação do tipo da variável ocorre no final da instrução, a usual prática de inicializar variáveis no momento da declaração não é um recurso suportado pela linguagem SAL.

A declaração de vetores segue as mesmas definições, tendo sua sintaxe na seguinte forma:

```
<identificador>[<tamanho>] : <tipo>;
```

Observe que em SAL os vetores são indexados por números inteiros, que variam de 1 até o valor correspondente ao tamanho estabelecido quando da definição (declaração) do vetor.

Nota: *Na memória, a posição correspondente ao índice 0 armazena o tamanho lógico do vetor, sendo utilizada no endereçamento de memória e no controle de acesso. A posição zero não pertence ao vetor do ponto de vista da linguagem, é um detalhe de implementação e nunca pode ser referenciada pelo código (não se encontra disponível e nem visível ao programador).*

Além de conhecer a sintaxe de declaração, é importante compreender onde essas declarações podem ocorrer dentro de um programa SAL, uma vez que isso determina o escopo e a visibilidade das variáveis. A linguagem organiza as declarações em duas seções distintas: `globals` e `locals`.

Variáveis declaradas na seção `globals`, localizada logo após o cabeçalho do módulo, possuem (como o nome sugere) um escopo global, isto é, podem ser acessadas e manipuladas em qualquer parte do programa, incluindo dentro de procedimentos e funções. Essas variáveis existem durante toda a execução do programa e são frequentemente utilizadas para armazenar informações de uso compartilhado entre diferentes sub-rotinas.

Por sua vez, a seção `locals` aparece exclusivamente no interior de sub-rotinas – sejam procedimentos ou funções – e serve para definir variáveis cujo uso é restrito àquele bloco específico. Variáveis locais têm escopo limitado à sub-rotina em que foram declaradas e deixam de existir quando sua execução é encerrada.

Essa separação clara entre variáveis globais e locais contribui para a organização do código, reduz a probabilidade de conflitos de nomes e reforça boas práticas de encapsulamento.

Ambas as seções são opcionais e independentes, permitindo ao programador estruturar o armazenamento de dados de acordo com as necessidades lógicas de cada programa.

Delimitadores

Alguns caracteres têm função específica dentro de uma linguagem e determinam a forma como são interpretados pelo compilador. Em SAL os caracteres abaixo são utilizados para marcar o início e o fim os seguintes elementos de programação:

"..." Strings
'...' Caracteres

Comentários

Suportar textos explicativos curtos ou trechos de documentação mais longos é uma característica básica e comum as linguagens de programação. SAL especifica duas formas para a documentação de código: comentários de uma única linha ou de múltiplas linhas.

@{ ... }@ Bloco (múltiplas linhas)
@ ... Linha (até o fim da linha)

Palavras reservadas

Reunimos a seguir todas as palavras que apresentam algum significado dentro da linguagem e que não devem ser utilizadas como identificadores pelo programador:

```
bool, char, do, else, end, false, fn, for, globals, if,
int, locals, loop, match, module, otherwise, print,
proc, ret, scan, start, step, to, true, until, when,
while
```

Operadores

Todas as operações suportadas pela linguagem, quer sejam matemáticas (lógicas e aritméticas) ou de armazenamento (atribuição), são representadas por operadores específicos. Em geral, as linguagens de programação definem um conjunto de símbolos - formados por um ou mais caracteres - que determinam a operação e a quantidade de operandos envolvidos nela.

A seguir encontrarmos a listagem, agrupados por categoria/natureza, dos operandos e operações suportadas pela linguagem SAL:

Categoria	Operador	Operação
Atribuição	:=	Armazena valor em variáveis
Aritméticos	+	Adição
	-	Subtração

	*	Multiplicação
	/	Divisão de inteiros
Relacionais	=	Igualdade
	<>	Diferença
	>	Maior
	<	Menor
	>=	Maior ou igual
	<=	Menor ou igual
Lógicos	^	Conjunção
	v	Disjunção
	~	Negação

Ordem de avaliação em expressões

Ao avaliar expressões, a linguagem SAL segue uma hierarquia rigorosa de precedência entre os operadores e que estabelece quais operações devem ser processadas primeiro. Isso garante a consistência na avaliação sem a necessidade do uso excessivo de parênteses, enquanto permite ainda que, desejando tornar o código mais inteligível, o uso explícito de parênteses seja feito.

A tabela a seguir resume a precedência de todos os operadores suportados pela linguagem, ordenados do mais prioritário para o menos prioritário. Cabe ainda destacar que todos os operadores binários são associativos à esquerda, enquanto operadores unários (tais como: ~), são associativos à direita.

Categoria	Operador(es)	Descrição / Observação
Unários	~, -	Negação lógica; negativo unário (associativos a direita)
Aritméticos	*, /	Multiplicação e divisão
	+, -	Adição e subtração binária
Relacionais	<, <=, >, >=	Comparações
	=, <>	Igualdade e diferença
Lógicos	^	Conjunção lógica
	v	Disjunção lógica (menor precedência entre os binários)

Isto significa que a expressão `a + b > c ^ d = e v ~f` seria avaliada na linguagem SAL na seguinte ordem:

1. `a + b` (adição)
2. `(resultado1) > c` (maior)
3. `d = e` (igualdade)
4. `(resultado2) ^ (resultado3)` (conjunção lógica)
5. `~f` (negação)
6. `(resultado4) v (resultado5)` (disjunção lógica)

Essa ordem produz uma leitura determinística da expressão, evitando ambiguidades. Finalmente, vale reforçar que parênteses podem ser usados para alterar explicitamente a ordem natural de avaliação, tornando a intenção do programador imediatamente evidente.

Blocos de comandos

Na linguagem SAL é possível agrupar várias linhas de comandos, definindo um bloco de comandos e que, em termos estruturais, se comporta como um elemento único. Para construí-lo usamos as palavras reservadas `start` e `end`, que identificam respectivamente o início e o fim do conjunto de comandos, dados da seguinte forma:

```
start
  <comandos que compõem o bloco >
end
```

Internamente, um bloco de comandos pode conter: nenhum, um ou vários comandos, ou atém mesmo, outros blocos de comandos. Vale ressaltar que um bloco de comando vazio, isto é, aquele que não contém nenhum comando, é um trecho de código estruturalmente correto e funcionalmente sem efeito.

Comandos de entrada e saída (*scan*, *print*)

A linguagem SAL apresenta um comando para realizar as operações de saída de dados em vídeo - `print` - e outro para a entrada de dados - `scan` -, que varre as informações a partir do teclado. Ambos são descritos com mais detalhes a seguir.

- *Comando PRINT:*

O comando `print` é capaz de escrever diferentes tipos de informação na tela, podem ser: literais, caracteres, strings, conteúdo de variáveis, resultados de expressões ou valores retornado de chamadas a sub-rotinas.

Sua utilização é dada no formato:

```
print( <elemento> ) ;
```

... ou ainda, por:

```
print( <elemento>, <elemento>, ... , <elemento> ) ;
```

Observe que a forma estrutural do comando suporta a escrita de múltiplos valores simultaneamente, ou seja, é possível escrever vários elementos na tela de uma só vez simplesmente separando-os por vírgulas.

- *Comando SCAN:*

Este comando recebe um dado a partir do teclado e o armazena na variável informada como parâmetro. A sintaxe do comando `scan` obedece a forma:

```
scan( <variável> ) ;
```

As informações do teclado são lidas e transferidas para endereços na memória, portanto, é esperado que apenas variáveis sejam informadas como argumentos nas chamadas de `scan`.

Diferentemente do comando de escrita, que suporta múltiplos argumentos, sempre que houver necessidade de ler mais de um dado do teclado, o programador deverá usar um comando `scan` para cada leitura.

Comandos de decisão (*if*, *match*)

Os comandos de desvio controlam o fluxo de execução, direcionando-o para algum outro trecho do código. O destino do desvio é determinado a partir de algum elemento de decisão, que em geral pode ser uma expressão lógica ou o valor de uma variável.

A linguagem SAL dispõe de dois comandos condicionais, um simples (`if`) e um composto (`if...else`); além de um comando de seleção (`match`).

Os comandos condicionais direcionam o fluxo de execução de acordo com o resultado da avaliação de uma expressão lógica. Em SAL as estruturas condicionais são construídas com o comando `if`, cuja sintaxe é:

```
if ( <expressão> )
  <comando ou bloco de comandos> ;
```

... ou ainda, na forma de um condicional composto, por:

```
if ( <expressão> )
  <comando ou bloco de comandos>
else
  <comando ou bloco de comandos> ;
```

Nota: *O emprego dos parênteses na expressão é obrigatório, fazendo parte da estrutura sintática do comando if.*

O seu funcionamento é simples e idêntico ao que se observa nas demais linguagens de programação. Sempre que a expressão é avaliada como verdadeira o fluxo de execução é direcionado para o comando ou bloco de comandos associado a seção `if`. Caso resulte em falso, o fluxo é direcionado para o primeiro comando subsequente a estrutura do condicional (nos condicionais simples); ou para o comando ou bloco de comandos associado a seção `else` (no caso de condicionais compostos).

Embora seja possível combinar condicionais em arranjos encadeados ou aninhados, dispor de um comando que chaveie o fluxo de execução entre múltiplos destinos possibilita a escrita de códigos mais curtos, legíveis e de fácil compreensão.

O comando `match` da linguagem SAL tem como propósito oferecer um mecanismo estruturado de desvios múltiplos, permitindo ao programador selecionar diferentes caminhos de execução com base no valor de uma variável ou expressão numérica.

Similar em função aos comandos *switch* de linguagens como C e Java, o comando `match` tem estruturação mais moderna, com uma sintaxe mais expressiva e flexível, suportando nativamente o uso de listas de valores e intervalos.

```
match ( <expressão> )
  when <valor1>                => <comando> ;
  when <valor2> , <valor3>     => <comando> ;
  when <valor4> .. <valor5>    => <comando> ;
  ...
  otherwise   => <comando> ;
end
```

A construção inicia sempre com `match(expr)`, onde *expr* é avaliada uma única vez; em seguida, cada cláusula `when` define um conjunto de condições possíveis, especificando valores exatos (ex: 1 ou 2, 3) ou intervalos numéricos (ex: 3..9). Quando a expressão se iguala a qualquer uma das alternativas indicadas em um `when`, o comando associado a essa cláusula é executado imediatamente.

O comando pode incluir uma cláusula `otherwise` (opcional), responsável por capturar todos os valores não cobertos pelos itens anteriores e que funciona como um "caminho padrão", garantindo que o programa tenha um comportamento definido mesmo quando nenhuma alternativa se encaixar.

Comandos de repetição (*for*, *loop while* e *loop until*)

Para realizar repetições a linguagem SAL dispõe de três estruturas de controle: o comando `for`, empregado quando a quantidade de iterações é pré-determinada; e os comandos `loop while` e `loop..until`, para situações em que condicionais governam e determinam quando o término das iterações deve ocorrer.

○ *Comando FOR*

O `for` realiza uma quantidade pré-estabelecida de repetições do comando ou bloco a ele associado. A estrutura do comando tem a seguinte forma:

```
for <varCtrl> := <valorIni> to <valorFim> do
   <comando ou bloco de comandos> ;
```

O funcionamento é regido por uma variável de controle, previamente declarada e cujo valor inicial é determinado por uma atribuição realizada na primeira seção do comando. A cada iteração, a variável de controle é incrementada até que o limite estabelecido pelo valor informado (dado entre `to` e `do`) seja extrapolado.

Por padrão, o incremento é feito de maneira progressiva e na razão de uma unidade (+1). No entanto é possível modificar o passo, estabelecendo outro fator de referência através do elemento `step`. Isso permite variar o fator de incremento com passos que sejam múltiplos de dois, três ou mais; ou definir uma contagem de forma decrescente empregando algum valor negativo como passo.

Assim, a sintaxe com todos os elementos suportados tem a forma:

```
for <varCtrl> := <valorIni> to <valorFim> step <fator> do
   <comando ou bloco de comandos> ;
```

○ *Comando LOOP WHILE*

Com uma estrutura mais simples que o anterior, o `loop while` é um comando de repetição condicional. Isto é, o processo iterativo é governado por uma expressão condicional e não por um contador.

O comando tem a seguinte estrutura sintática:

```
loop while ( <expressão> )
   <comando ou bloco de comandos> ;
```

Essa estrutura de repetição avalia a expressão condicional antes de executar o comando (ou bloco de comandos) a ela associada. Assim, a repetição ocorre somente se a expressão dentro dos parênteses resultar em verdadeiro.

Note que se acaso a condição for inicialmente falsa, o comando interno não será executado nenhuma vez, caracterizando o comportamento típico de um laço de pré-teste.

Ao final de cada iteração a expressão é novamente avaliada, determinando se uma nova repetição deve ocorrer ou se o fluxo seguirá para o próximo comando subsequente.

○ *Comando LOOP ... UNTIL*

Sendo também um comando condicional, o `loop...until` diferencia-se do `loop while` em dois aspectos: quanto a sintaxe e quanto ao critério de parada. Aqui a repetição continua enquanto a expressão for falsa; encerrando quando for verdadeira.

Sua sintaxe fica definida da seguinte maneira:

```
loop
   <comando ou comandos>
until ( <expressão> ) ;
```

Assim, por empregar duas palavras em sua estrutura sintática, torna-se dispensável a utilização das palavras `start` e `end` para demarcar um bloco de comandos.

Por avaliar a condição apenas ao término do bloco, os comandos localizados entre as palavras `loop` e `until` terão sido executados uma vez.

Neste ponto, o comando decidirá por realizar uma iteração se a condição avaliada ainda não tiver sido satisfeita, ou seja, resultar em falso.

Estruturação e seções de um programa em SAL

Os programas na linguagem SAL iniciam com a palavra reservada `module`, um nome que o identifica e sendo seguido do procedimento `main()`, que contém o código principal do programa. A seguir encontramos um exemplo com a estrutura mínima e que é comum a todos os programas:

```
module <nomePrg> ;

proc main()
start
   <comandos>
end
```

Em muitas situações os programas precisam manipular dados, quer sejam resultados parciais ou de ler informações do teclado, necessitando assim de variáveis para armazená-los temporariamente. Como dito anteriormente, as variáveis podem ter escopo global ou local, dependendo do local em que são definidas.

A existência das seções `globals` e `locals`, bem como a de quaisquer sub-rotinas além da main, é facultativa. Assim, podemos descrever a estrutura de um programa genérico como um modelo mais completo, evidenciando o local de cada seção, como se segue:

```
module <nomePrg> ;

globals
   @declarações de variáveis globais

@definição de outras sub-rotinas

proc main()
locals
   @variáveis locais
start
   @bloco de comandos principal>
end
```

Sub-rotinas (procedimentos e funções)

Como é sabido a linguagem SAL suporta a criação de sub-rotinas, sendo definidas em uma região específica do código fonte: entre as variáveis globais e o procedimento principal (`main`).

Habitualmente as sub-rotinas são conhecidas como procedimentos ou funções, diferenciam-se essencialmente como aquelas que não retornam valores (`proc`) daquelas que devolvem algum valor ao ponto de chamada (`fn`). Assim há uma ligeira diferença na assinatura de cada uma delas:

```
proc <nomeProcedimento> ( <lstParams> )

fn <nomeFunção> ( <lstParams> ): <tipoRetorno>
```

Em SAL uma função pode retornar apenas uma informação, compatível algum dos tipos básicos da linguagem. Se a sub-rotina não retorna qualquer valor ao ponto de chamada, deverá ser construída como um procedimento e não uma função.

Quanto aos nomes, o programador pode utilizar qualquer identificador que seja válido na linguagem. Contudo, é sugerida a escolha de um termo condizente com a tarefa realizada pela sub-rotina.

Frequentemente sub-rotinas precisam considerar dados externos para a execução de suas tarefas. Embora a linguagem SAL suporte a definição de variáveis globais, o mais comum é definir parâmetros cujos valores serão associados no momento de chamada. A lista de parâmetros de uma sub-rotina é estabelecida em seu cabeçalho, no momento de sua definição. Assim, toda sub-rotina que não necessitar de parâmetros deverá definir uma lista vazia em seu cabeçalho, enquanto as demais devem seguir os seguintes critérios:

- Cada parâmetro deve ter um tipo associado e ser separado dos demais por uma vírgula. Caso tenhamos mais do que um parâmetro do mesmo tipo, cada um deles deve especificá-lo novamente.
- O valor dos parâmetros deve ser informado no momento da chamada, respeitando-se a ordem e compatibilidade de tipos especificados no cabeçalho da função, quando da sua definição.
- Por se apresentarem funcionalidade similar a das variáveis locais, os parâmetros estão sujeitos as mesmas regras de escopo.

Nota: *a passagem de parâmetros por referência não é suportada nesta versão da linguagem. No entanto, variáveis globais podem ser manipuladas e modificadas em qualquer sub-rotina.*

Tal como ocorre no procedimento principal, uma sub-rotina pode ter uma seção para a declaração de variáveis locais. Podemos dizer então que a sintaxe de uma função, por exemplo, teria a forma:

```
fn <nome> ( <lstParams> ): <tipoRet>
locals
   @declaração de variáveis locais
start
   @comandos da sub-rotina
   ret @valorRetornado ;
end
```

Exemplos de programas em SAL

O programa mais simples e mais comum quando da introdução de uma nova linguagem é o “Olá Mundo!”. Assim, nosso primeiro exemplo será a sua codificação em SAL.

```
module Exemplo_01;

proc main()
start
  print("Olá mundo!");
end
```

Mesmo os programas mais simples, realizam algum tipo de entrada e saída de dados, portanto o exemplo que se seque declara uma variável do tipo int para armazenar um valor inteiro digitado pelo usuário e escreve o valor digitado na tela.

```
module Exemplo_02;

proc main()
locals
   x: int;
start
  print("Entre com um número inteiro:");
  scan(x);
  print("O número que você digitou é:",x);
end
```

Neste exemplo, temos um programa que define e invoca uma sub-rotina chamada soma, responsável por realizar a adição dos dois valores inteiros

que lhe sejam passados como parâmetro, escrevendo o resultado de seu processamento no vídeo.

```
module EXEMPLO_03;

@função que soma dois valores
fn SOMA(A: int, B: int): int
start
    ret A+B;
end

@programa principal
proc main()
start
   print(SOMA(1,2));
end
```

O problema de verificar se, dadas quaisquer três medidas inteiras, elas formam um triângulo e de qual tipo ele seria, é um exemplo interessante para demonstrar a questão do aninhamento de comandos if's, bem como, a utilização dos operadores lógicos nas expressões. Assim, temos:

```
module Exemplo_04;

proc main()
locals
   a,b,c: int;
start
   print("Entre com três valores inteiros:");
   scan(a);
   scan(b);
   scan(c);
   if (a+b < c v a+c < b v b+c < a)
     print("Essas medidas não formam um triângulo.")
   else
     if (a = b ^ b = c)
       print("As medidas formam um triângulo equilátero")
     else if ( a <> b ^ b <> c)
            print("As medidas formam um triângulo escaleno")
          else
            print("As medidas formam um triângulo isósceles");
end
```

Um bom exemplo da utilização do comando loop while é o processo de validação de dados de entrada. Suponha que queremos ler um número inteiro digitado pelo usuário e só prosseguir quando esse número estiver no intervalo [1, 100]. Poderíamos implementar a seguinte solução usando linguagem SAL:

```
module Exemplo_05;

proc main()
locals
   x : int;
start
   print("Digite um inteiro entre 1 e 100:");
   scan(x);
   loop while (x < 1 v x > 100)
   start
      print("Valor invalido. Digite novamente (1..100):");
      scan(x);
   end;
   print("Valor aceito:", x);
end
```

Suponha um problema onde, dada uma progressão aritmética (PA) com primeiro termo A1 e razão R, queiramos calcular o somatório dos n primeiros termos.

Talvez uma forma direta e didática, seria calcula cada termo (o *i*-ésimo termo é $A_1 + (i - 1) * R$) e somarmos os elementos de i = 1 até *n*. Essa estratégia ilustra bem o emprego do for, que nos permite definir o laço com limites explícitos e, se desejado, controlamos o passo.

```
module Exemplo_06;

proc main()
locals
   A1, R, n, i, soma, termo : int;
start
   print("Primeiro termo (A1):"); scan(A1);
   print("Razão (R):");           scan(R);
   print("Qtde de termos (n):");  scan(n);
   soma := 0;
   for i := 1 to n do
   start
      termo := A1 + (i - 1) * R;
      soma := soma + termo;
   end ;
   print("Somatório dos ", n, " primeiros termos da PA: ", soma);
end
```

Imagine um programa que determine a prioridade de chamados recebidos em que, dado um código de chamado digitado pelo usuário, o programa classifique sua prioridade segundo uma tabela e reporte a ação adequada. Essa é uma situação interessante para exemplificar o uso comando match e, se considerarmos que há casos pontuais, listas de valores e intervalos para os códigos, podemos ilustrar as diferentes formas da cláusula when.

```
module Exemplo_07;

proc main()
locals
   cod : int;
start
   print("Digite o código do chamado:");
   scan(cod);
   match (cod)
      when 0        => print("Teste: nenhuma ação necessária.");
      when 9999     => print("Encerrar sistema de triagem.");
      when 1, 2, 3  => print("URGENTE: acionar equipe de resgate.");
      when 100..199 => print("Prior. Alta: encaminhar ao n2.");
      when 200..499 => print("Prior. Média: direcionar ao n1.");
      when 500..999 => print("Prior. Baixa: colocar na fila.");
      otherwise     => print("Código inválido ou desconhecido.");
   end;
end
```

Apêndice A: EBNF da Linguagem SAL

Sabendo que na EBNF os tokens iniciados com “s” representam categorias lexicais, não os lexemas literais da linguagem, podemos definir formalmente a linguagem através da seguinte gramática:

```
G = (N, Σ, P, ini)
P:   ini   ::= “sMODULE” id ";“ glob? subs? princ
     id    ::= “sIDENTIF”
     glob  ::= “sGLOBALS” decls+
     decls ::= id (",“ id)* ":“ tpo ";“
     tpo   ::= “sINT” | “sBOOL" | “sCHAR" | tpo ”[“ ”sCTEINT“ ”]“
     vec   ::= id ”[“ (”sCTEINT“ | id) ”]“
     subs  ::= (func | proc)+
     func  ::= “sFN” id “(” param? ”)” ”:” tpo bco
     proc  ::= “sPROC” id “(” param? ”)” bco
     princ ::= “sPROC” “sMAIN” “(“ “)” bco
     param ::= id ":“ tpo (",“ id ":“ tpo)*
     bco   ::= “sSTART” (cmd ";“)* ”sEND”
     cmd   ::= out | inp | if | mat | fr | wh | rpt |
                atr | call | ret | bco
     out   ::= “sPRINT” ”(“ elem (”,“ elem)* ”)“
     inp   ::= “sSCAN” ”(“ (id | vec) ”)“
     if    ::= “sIF” “(“ expr ) cmd (”sELSE” cmd)?
     mat   ::= "sMATCH" "(" expr ")" wlist "sEND"
     wlst  ::= whn+ (othr)?
     whn   ::= "sWHEN" wcnd "sIMPLIC" cmd ";"
     othr  ::= "sOTHERWISE" "sIMPLIC" cmd ";"
     wcnd  ::= witem (",” witem )*
     witem ::= wint | wrnge
     wrnge ::= wint "sPTOPTO" wint
     wint  ::= ("sSUBRAT")? "sCTEINT"
     fr    ::= “sFOR” atr “to“ (“sSTEP“ (id |“sCTEINT“))? “do“ cmd
     wh    ::= “sLOOP“ “sWHILE” “(“ expr “)“ cmd
     rpt   ::= “sLOOP” (cmd “;“)* “sUNTIL” “(“ expr “)“
     atr   ::= (id | vec) “sATRIB” elem
     call  ::= id “(“ (expr (“,“ expr)* )? “)“
     ret   ::= “sRETURN” elem
     elem  ::= litl | id | vec | call | expr
     litl  ::= “sSTRING” | “sCTEINT” | “sCTECHAR”
     expr  ::= exlog (”sOR” exlog)*
     exlog ::= exrel (”sAND” exrel)*
     exrel ::= exari (oprel exari)*
     exari ::= exarp (opari exarp)*
     exarp ::= fact (oparp fact)*
```

```
fact  ::=  elem | “sNEG“ ftr | “sSUBRAT“ ftr | “(“expr“)“
oprel ::= “sMAIOR” | “sMAIORIG” | “sIGUAL” |
          “sMENOR” | “sMENORIG” | “sDIFERENTE”
opari ::= “sSOMA“ | “sSUBRAT“
oparp ::= “sMULT“ | “sDIV“
```